



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS TRINDADE
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CURSO CIÊNCIAS DA COMPUTAÇÃO

Leonardo Fonseca Franchini - 23100484
Marcos Vinicius Moreira Machado - 23100478
Rita Louro Barbosa – 22203157

SO2

Entrega 4

1 Introdução

Este relatório descreve o progresso obtido na quarta etapa do projeto proposto pela disciplina de Sistemas Operacionais II.

Na Entrega 1, foram abordados a instalação e o preparo da infraestrutura necessária para viabilizar a comunicação entre sistemas independentes, cada um representado por uma máquina virtual QEMU, utilizando raw sockets para a transferência de frames Ethernet sem IP.

Na Entrega 2, foi abordada a implementação da comunicação entre componentes de um mesmo sistema autônomo (intra-VM), utilizando memória compartilhada no modelo Produtor × Consumidor com IPCs System V (shmget e semget), por meio da classe ShmEngine. Além disso, foi introduzida a multiplexação de portas para identificação dos componentes (sensores, atuadores, powertrain, gateway) dentro de uma mesma VM.

Na entrega 3, o objetivo foi garantir que todos os veículos compartilhem uma mesma noção de tempo, de forma que cada mensagem trocada na rede possa ser identificada de forma inequívoca pela combinação do seu endereço de origem com o instante em que foi gerada. Para isso, foi implementada uma versão simplificada do Precision Time Protocol (PTP – IEEE 1588), na qual uma unidade de infraestrutura externa chamada RSU (Road Side Unit) atua como referência de tempo para todos os veículos.

O foco desta entrega 4 é garantir que os sistemas autônomos consigam sincronizar suas posições relativas e se comunicar apenas com agentes do mesmo quadrante espacial. Para isso, foram implementados: um módulo de kernel que expõe as coordenadas de cada VM via `/proc/position`; a etiquetagem das mensagens com o quadrante de origem; e a expansão da infraestrutura de RSUs para quatro unidades, uma por quadrante, com troca automática de RSU conforme o veículo se desloca. Além disso, esta entrega inclui a correção de erros identificados na etapa anterior.

2 Objetivos da P4

O principal objetivo desta entrega é capacitar os sistemas autônomos a sincronizar suas posições relativas e filtrar mensagens por quadrante espacial, garantindo que componentes de sistemas distintos não interfiram entre si. Para isso, os seguintes itens foram desenvolvidos:

- Implementação do módulo de kernel `position.ko`, que cria a interface `/proc/position` e expõe o quadrante atual da VM (0–3), atualizado por um timer a cada 4 s.
- Etiquetagem das mensagens com coordenadas espaciais na origem: o campo `src_quadrant` é preenchido em `Component::send()` via `Position::quadrant()` e propagado no cabeçalho de cada frame.

- Filtragem de mensagens por quadrante na NIC via cross-layer design: frames cujo `src_quadrant` difere do quadrante local são descartados pela NIC antes de chegar ao Protocol, garantindo isolamento entre sistemas autônomos distintos.
- Expansão para quatro RSUs (uma por quadrante), com troca automática de RSU conforme o veículo se desloca entre quadrantes.

3 Implementação

3.1 Informações gerais

O programa opera com múltiplas VMs simulando carros autônomos. O processo principal de cada VM — o gateway — inicializa as interfaces de rede e o protocolo de comunicação e, em seguida, cria processos filhos via `fork()` que simulam os componentes do veículo: sensor, atuador, powertrain e TimeClient. A partir desta entrega, cada VM carrega o módulo de kernel `position.ko`, que expõe o quadrante atual via `/proc/position`. Todos os componentes da mesma VM leem esse arquivo, garantindo percepção compartilhada e consistente do espaço.

Cada componente instancia seu próprio Communicator, que encapsula o protocolo e o endereço lógico do componente (par MAC + porta). O envio e recebimento de mensagens é feito através dessa instância, que utiliza o Protocol para empacotar e desempacotar dados e gerenciar o roteamento entre NICs. A partir desta entrega, todas as mensagens passam a carregar um campo `timestamp` preenchido via `clock_gettime(CLOCK_REALTIME, ...)` no momento do envio, com precisão em nanossegundos. O formato das mensagens é `M = {address, timestamp, payload}`.

3.2 Classes

Nesta seção detalharemos as classes do sistema, como elas funcionam e se relacionam, com ênfase nas novidades desta entrega.

3.2.1 RawSocketEngine

A classe `RawSocketEngine` é responsável pelo acesso de baixo nível ao hardware de rede, utilizando sockets RAW (`AF_PACKET / SOCK_RAW`) para enviar e receber frames Ethernet diretamente, sem passar pela pilha IP do sistema operacional. Durante sua inicialização, ela abre o socket com o EtherType customizado `0x8888`, obtém o índice e o endereço MAC da interface via `ioctl`, realiza o `bind` na interface e configura um timeout de 100 ms para que o loop de recepção não bloqueie indefinidamente.

A engine lança uma thread dedicada que executa o loop de recepção em segundo plano. Quando um frame chega e passa pelo filtro de EtherType, essa thread chama `_handle`, implementado pela `NIC<RawSocketEngine>`, que copia o frame para um buffer do pool e notifica os observadores registrados. O envio é feito pelo método `_send`, que monta uma estrutura `sockaddr_ll` com o endereço de broadcast e transmite o frame via `sendto`.

3.2.2 NIC

A classe NIC é o núcleo da comunicação física do sistema, implementada como um template sobre uma engine (NIC<E>), combinando a interface abstrata NICBase com a engine concreta via herança privada. Essa separação permite reutilizar a mesma lógica de gerenciamento de buffers, endereçamento e notificação de observadores tanto para RawSocketEngine quanto para ShmEngine, apenas trocando o parâmetro de template.

A NIC mantém um pool interno de buffers para frames, evitando alocações dinâmicas frequentes. Ela implementa o padrão Observer, permitindo que protocolos se registrem para receber notificações de novos frames por EtherType. Um mutex protege o pool contra acesso concorrente entre a thread de recepção e o processo principal.

3.2.3 Protocol

A classe Protocol é responsável por empacotamento, desempacotamento e roteamento interno das mensagens. Ela define o formato dos pacotes adicionando um cabeçalho lógico (Protocol::Header) com portas de origem e destino, tamanho do payload e o campo src_quadrant, preenchido com o quadrante atual lido de /proc/position no momento do envio. O endereço MAC de origem é extraído diretamente do frame Ethernet, evitando redundância com o cabeçalho de enlace.

O Protocol observa a NIC e, ao receber um frame com o EtherType correto, executa o filtro de quadrante antes de qualquer roteamento. Apenas frames do mesmo quadrante prosseguem para extração da porta de destino e notificação do Communicator registrado. Para o envio, o Protocol seleciona a NIC adequada comparando expected_dst() com o destino — mensagens inter-VM saem pelo raw socket e mensagens intra-VM saem pela memória compartilhada, de forma transparente para as camadas acima.

Para suporte ao unicast nas respostas PTP, o filtro de seleção de NIC foi refinado: a comparação com expected_dst() só é aplicada quando o destino não é vazio, permitindo que a RawSocketEngine aceite tanto destinos broadcast quanto unicast de forma transparente. O destino é broadcast — todas as RSUs recebem, e apenas a do quadrante correto aceita pelo filtro de src_quadrant.

3.2.4 Communicator

A classe Communicator é a principal interface de comunicação para os componentes do veículo. Ao ser instanciado, ele se registra no Protocol para uma porta específica, garantindo que apenas mensagens destinadas a ele sejam recebidas. Ele mantém uma lista interna de mensagens e um semáforo para sincronização.

O método recebe bloqueia a thread chamadora por até 1 segundo aguardando a chegada de um novo dado. Quando update é chamado assincronamente pela thread de recepção, os dados do frame são copiados para um objeto Message alocado em heap, inserido na lista interna, e o semáforo é liberado, acordando a thread bloqueada. Esse mecanismo promove o processamento desacoplado e assíncrono das mensagens.

3.2.5 Message

A classe Message representa a unidade de dados trocada entre os agentes. Ela possui os campos `_data` (buffer de até 1024 bytes), `_size` (tamanho válido dos dados), `_src_mac` (endereço MAC de origem, preenchido pelo Communicator), `_origin` (quadrante de origem, 2 bits, preenchido via `set_origin()` com o valor de `Position::quadrant()` no momento do envio) e `_timestamp` (`uint64_t` em nanossegundos, preenchido com `clock_gettime(CLOCK_REALTIME, ...)`). O formato é `M = {origin, timestamp, payload}`, conforme especificado na entrega.

3.2.6 PTPFrame

A struct PTPFrame representa o payload das mensagens de sincronização temporal. Ela é composta por `message_type` (`uint8_t`), que indica a etapa do fluxo PTP, e `timestamp` (`uint64_t`), com o instante de tempo relevante para aquela etapa. Os valores de `message_type` são:

Valor	Tipo	Descrição
0	Sync Request	Enviado pelo slave para iniciar a sincronização
1	Sync Response	Enviado pelo master com o timestamp T1
2	Delay_Req	Enviado pelo slave para medir o atraso de rede
3	Delay_Resp	Enviado pelo master com o timestamp T4

A struct também contém o campo `seq` (`uint32_t`), um número de sequência único por veículo gerado a partir do último byte do MAC (`mac.bytes[5] << 24 | counter`), que permite ao veículo correlacionar cada resposta da RSU com o request correspondente, descartando respostas destinadas a outros veículos.

A struct é marcada com `__attribute__((packed))` para garantir ausência de padding, permitindo que seja copiada diretamente para o payload de um Message e transmitida pela rede sem inconsistências.

3.2.7 RSU (Road Side Unit)

Nesta entrega, o sistema passa a operar com quatro RSUs, uma por quadrante. Cada instância é executada em uma VM dedicada com um MAC fixo. Cada RSU recebe seu quadrante como parâmetro no construtor, passado via linha de comando (`quadrant=N` no `cmdline` do kernel). Ao inicializar, chama `Protocol::set_quadrant(_quadrant)`. A classe RSU cumpre o papel de master PTP e possui uma `NIC<RawSocketEngine>`, um Protocol e um Communicator registrado

na porta reservada 9999. Ao receber uma mensagem, a RSU verifica se o quadrante do remetente corresponde ao seu; caso contrário, descarta a mensagem. Para mensagens válidas, age conforme o `message_type`:

- **Sync Request (0):** registra o instante atual como T1 e responde com `{type=1, timestamp=T1}`.
- **Delay_Req (2):** registra o instante de recebimento como T4 e responde com `{type=3, timestamp=T4}`.

As respostas são enviadas em unicast diretamente para o MAC do veículo que originou o request, extraído via `msg.src()` do frame Ethernet recebido. Isso evita flooding na rede e segue o modelo unicast on-demand definido pelo IEEE 1588-2008 Annex D, recomendado para redes veiculares onde broadcast não escala.

3.2.8 TimeClient

O TimeClient é o componente de cada veículo responsável por sincronizar o relógio da VM com a RSU e disponibilizá-lo aos demais componentes. Ele herda de Component (possuindo uma NIC<ShmEngine> para comunicação intra-VM) e possui adicionalmente uma NIC<RawSocketEngine> própria, anexada ao mesmo Protocol, para se comunicar diretamente com a RSU pela rede.

O método `syncTime` implementa o fluxo PTP do lado slave:

1. Envia `PTPFrame{type=0}` para a RSU e aguarda resposta.
2. Ao receber `{type=1, T1}`, registra o instante de recebimento como T2.
3. Envia `PTPFrame{type=2}` (Delay_Req) e registra o instante de envio como T3.
4. Ao receber `{type=3, T4}`, calcula:
 - $\text{delay} = ((T2 - T1) + (T4 - T3)) / 2$
 - $\text{offset} = ((T2 - T1) - (T4 - T3)) / 2$
5. Ajusta o relógio local via `clock_settime(CLOCK_REALTIME, ...)` somando o offset ao tempo atual.

Em vez de aplicar o offset inteiro de uma vez — o que causaria saltos abruptos no relógio e corromperia medições futuras — aplica-se apenas `correction = -offset / 8` a cada ciclo, implementando um PLL (Phase-Locked Loop) simples que converge gradualmente.

O método `run` repete esse ciclo a cada 500 ms. Como todos os processos de uma mesma VM compartilham o relógio do sistema operacional, o ajuste feito pelo TimeClient via `clock_settime` é imediatamente visível para todos os outros componentes daquele veículo, sem necessidade de comunicação adicional.

3.3 Fluxo de dados

Envio

O componente preenche um objeto Message com dados, tamanho, endereço de origem e timestamp, e o repassa ao Communicator. Este chama `Protocol::send`, que seleciona a NIC adequada com base no destino, aloca um buffer do pool, monta o

cabeçalho de protocolo e copia o payload. A engine correspondente — RawSocketEngine para inter-VM ou ShmEngine para intra-VM — realiza a transmissão efetiva. Após o envio, o buffer é devolvido imediatamente ao pool.

Recebimento

Na RawSocketEngine, uma thread dedicada aguarda frames no socket com timeout, filtra pelo EtherType 0x8888 e os entrega à NIC via `_handle`. Na ShmEngine, a chegada de um novo frame é sinalizada por SIGUSR1, disparado pelo processo que escreveu na memória compartilhada (ver seção 3.4).

Em ambos os casos, a NIC copia o frame para um buffer livre do pool e notifica o Protocol via `Observed::notify`. O Protocol extrai a porta de destino e notifica o Communicator registrado. O Communicator copia os dados para um Message em heap, insere na lista interna e libera o semáforo, desbloqueando a thread em receive. Após a extração, o buffer é devolvido ao pool.

3.4 Signal Handler

O tratamento de sinais é o mecanismo de notificação assíncrona da comunicação intra-VM. Quando um processo escreve um frame no ShmChannel, ele itera sobre os PIDs dos leitores registrados e envia `kill(pid, SIGUSR1)` para cada um, exceto para si mesmo.

O handler estático `_signal_handler` acessa a instância da ShmEngine via ponteiro global e chama `_try_read`, que lê todos os frames pendentes comparando o head individual com o tail global. Cada frame lido é entregue via `_handle`. O handler é registrado com `sigaction`, que permite definir com precisão a função chamada, os sinais bloqueados durante a execução e as flags de comportamento, garantindo tratamento seguro e previsível.

3.5 ShmEngine

A ShmEngine implementa a comunicação intra-VM usando memória compartilhada System V no modelo Produtor-Consumidor. No construtor, cria ou abre um segmento de memória via `shmget/shmat` e um semáforo System V via `semget` para proteger a seção crítica de escrita. O processo se registra como leitor no ShmChannel via `enroll(getpid())`, que atribui um slot no array de PIDs e inicializa o ponteiro de leitura individual.

O ShmChannel implementa uma fila circular de 8 slots de frames Ethernet. Cada leitor possui seu próprio head, de modo que múltiplos processos possam ler de forma independente sem interferência — equivalente a multicast dentro da VM. O tail, compartilhado entre todos, avança a cada novo frame inserido.

3.6 Endereçamento

As portas são definidas estaticamente no namespace Ports. O endereço completo de cada agente é o par {MAC, porta}, estruturado como Protocol::Address. Como o MAC é único por VM e a porta é única por tipo de componente dentro da VM, esse par identifica cada agente de forma globalmente única, sem depender de PIDs — que não cruzam fronteiras de VM e mudam a cada execução.

Componente	Porta
Gateway	1000
Sensor	1001
Actuator	1002
Powertrain	1003
TimeClient	1004
RSU	9999

3.7 Precision Time Protocol (PTP)

O Precision Time Protocol (PTP), definido pelo padrão IEEE 1588, é um protocolo de sincronização de relógios projetado para redes de computadores. Seus principais conceitos são:

- **Dois tipos de entidade:** mestre (*master*) e escravo (*slave*). O mestre é a referência de tempo da rede; os escravos ajustam seus relógios com base nas mensagens trocadas com ele.
- O protocolo funciona por troca de mensagens com timestamps: o mestre envia o instante em que transmitiu a mensagem (T1), o escravo registra o instante em que a recebeu (T2), depois envia uma requisição de delay e registra o instante de envio (T3), e o mestre registra o instante em que a recebeu (T4).
- Com os quatro timestamps, o escravo consegue calcular o atraso de propagação da rede e o offset entre seu relógio e o do mestre, ajustando seu relógio local de forma precisa.
- A escolha de qual agente atua como mestre pode ser arbitrária. Neste projeto, a RSU cumpre esse papel.

Nesta implementação, o protocolo foi simplificado para o contexto do projeto, mantendo o fluxo de quatro timestamps (Sync, Sync Response, Delay_Req e Delay_Resp) e o cálculo de offset e delay, mas sem os mecanismos de eleição de mestre e anúncio periódico presentes no padrão completo.

3.8 Comunicação entre VMs, sincronização temporal e troca de RSU

A comunicação inter-VM é realizada pela RawSocketEngine usando frames Ethernet raw com EtherType 0x8888, sem camada IP. A partir desta entrega, os veículos enviam sempre em broadcast. O TimeClient envia mensagens PTP com `src_quadrant` atualizado; a RSU do quadrante correspondente responde e as demais descartam pelo filtro da NIC. O Protocol garante o roteamento correto entre RawSocketEngine e ShmEngine de forma transparente para as camadas acima, conforme descrito na seção 3.2.3.

A sincronização temporal e a localização espacial integram esses mecanismos: o TimeClient usa a RawSocketEngine para trocar mensagens PTP com a RSU do quadrante atual, e o resultado do ajuste via `clock_settime` é automaticamente compartilhado com todos os componentes da mesma VM pelo relógio do sistema operacional. A troca de RSU é implícita. O veículo envia sempre em broadcast com `src_quadrant` atualizado via `Position::quadrant()`. A NIC de cada RSU filtra pelo quadrante via `Protocol::verifica_quadrante()`, aceitando apenas frames do seu quadrante. Não existe handoff explícito.

3.9 Módulo de kernel: position.ko

O módulo de kernel `position.ko` implementa a interface de sincronização espacial exigida pela entrega. Ao ser carregado (`insmod`), ele cria a entrada `/proc/position` com permissão somente-leitura (0444) e configura um timer do kernel com período de 4 segundos. A cada disparo do timer, o campo `quadrante` (`uint8_t`) é atualizado com um valor aleatório entre 0 e 3, obtido via `get_random_u32_below(4)`. A leitura é servida pelo handler `proc_read`, que copia o valor atual para o buffer do usuário via `copy_to_user`. O módulo é descarregado via `rmmod`, que cancela o timer com `timer_delete_sync` e remove a entrada `/proc`.

O acesso ao módulo pelo userspace é encapsulado pela classe `Position` (`include/utls/position.hpp`), que abre `/proc/position`, lê o inteiro e retorna `q & 0x3`. Como todos os processos de uma mesma VM leem o mesmo arquivo no kernel, a percepção do quadrante é idêntica para todos os componentes daquele sistema autônomo, cumprindo o requisito de percepção espacial compartilhada.

3.10 Cross-layer design:

A NIC e o Protocol implementam um design cross-layer: a NIC acessa o método privado estático `Protocol::verifica_quadrante()` via declaração `friend`, inspecionando o `src_quadrant` do `Protocol::Header` sem expor esse método publicamente. Isso permite que a filtragem ocorra na camada mais baixa possível (NIC, na subida), enquanto o Protocol mantém controle sobre a lógica de quadrante via `set_quadrant`, `get_quadrant` e `accept`.